Fraud Detection Case Study: Identifying Credit Card Fraud Transactions

Christina Qawwas

June 2026

Abstract

This report presents an end-to-end machine learning pipeline for detecting fraudulent credit card transactions on a severely imbalanced dataset of **594,643** transactions, where only **1.2%** are fraudulent. Given this imbalance, the pipeline is evaluated on **AUC-ROC and F1-score** with particular emphasis on fraud recall, as missing a fraudulent transaction represents a direct financial loss for the bank. The pipeline covers data preprocessing, exploratory data analysis, feature engineering, and model training. Three models were compared — Logistic Regression, Random Forest, and XGBoost. **SMOTE** was applied to the training data to address class imbalance, followed by hyperparameter tuning using **RandomizedSearchCV**. The final model achieved an **AUC-ROC of 0.9922** and a **fraud recall of 76%**, outperforming all baselines. Feature importance analysis identified transaction amount and merchant category as the strongest predictors of fraud.

Contents

1 Challenge

Fraud detection is one of the hardest ML problems because:

```
Total transactions:     594,643
Legitimate:             587,443  (98.8%)
Fraudulent:               7,200   (1.2%)
```

A model that predicts "not fraud" for every single transaction would be **98.8%** accurate — yet completely useless. This is why we never use accuracy as our metric.
We use:
AUC-ROC → how well the model ranks fraud risk
F1-Score → balance between catching fraud and avoiding false alarms
Recall → what % of actual fraud cases we catch (most critical for a bank)

2 Dataset

594,643 real credit card transactions with features: customer, age, gender, merchant, category, amount, and a fraud label. Raw data had quote-wrapped strings, unknown values, zero-variance columns, and ID fields — all handled in preprocessing.

Table 1: Dataset Features Description

Column	Description
`step`	Hour-based time unit (1 step = 1 hour)
`customer`	Unique customer ID
`age`	Age group (0 = ≤18, 1 = 19–25, ..., 6 = >65, U = Unknown)
`gender`	M / F / E (Enterprise) / U (Unknown)
`zipcodeOri`	Customer zip code
`merchant`	Merchant ID
`zipMerchant`	Merchant zip code
`category`	Merchant category (15 types)
`amount`	Transaction amount in EUR
`fraud`	**Target variable** — 0 = legitimate, 1 = fraud

3 Exploratory Data Analysis

Before training any model, we analyzed the data to understand fraud patterns across different dimensions. All charts below are generated directly from the dataset.

3.1 Class Imbalance

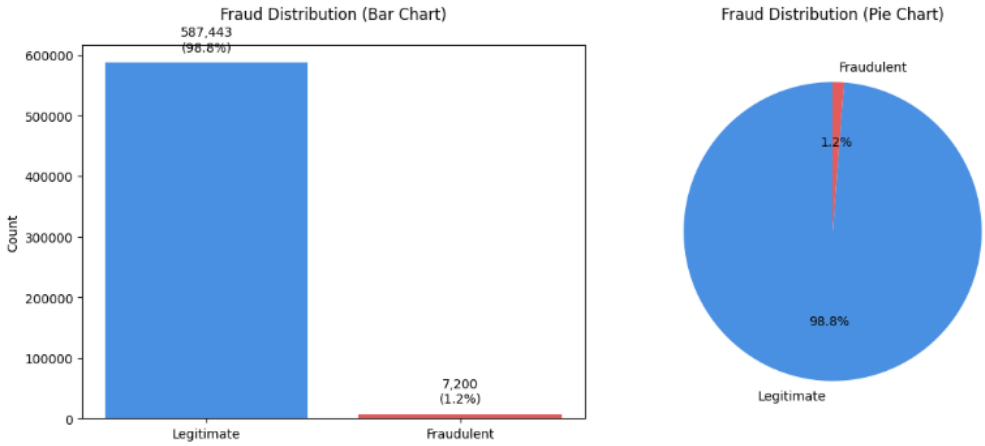

Figure 1: Distribution of legitimate vs. fraudulent transactions.

The dataset contains **594,643 transactions**, of which only **7,200 (1.2%) are fraudulent**. A model that blindly predicts every transaction as legitimate would score 98.8% accuracy — yet fail completely at its actual job — which is why accuracy is not used as an evaluation metric. This imbalance is later addressed using SMOTE.

3.2 Fraud Rate by Merchant Category

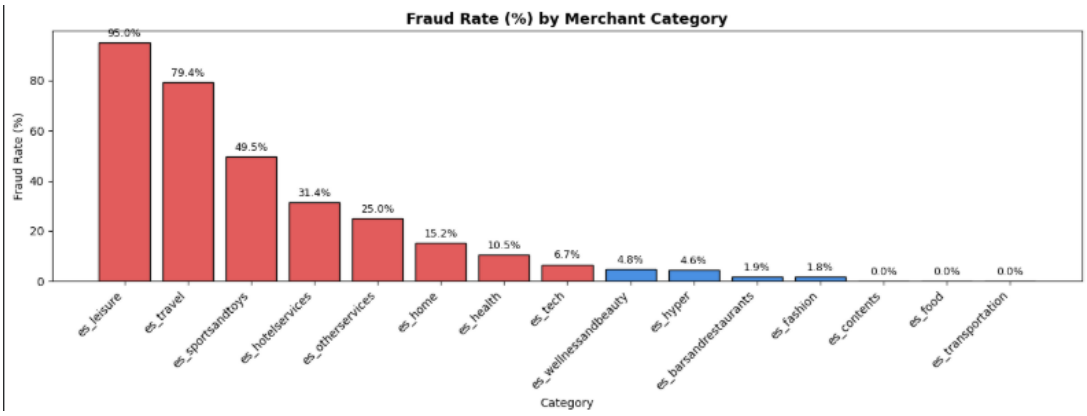

Figure 2: Fraud rate (%) per merchant category.

Fraud is heavily concentrated in specific categories. `es_leisure` (95%) and `es_travel` (79.4%) are by far the highest-risk categories, while `es_food` and `es_transportation` show virtually no fraud. This suggests the bank should apply stricter monitoring rules to leisure and travel transactions.

3.3 Fraud Rate by Gender

Female cardholders show the highest fraud rate at **1.47%**, compared to 0.91% for male and 0.41% for unknown/enterprise accounts. On its own this is not enough to act on, but it becomes relevant when combined with transaction amount or category.

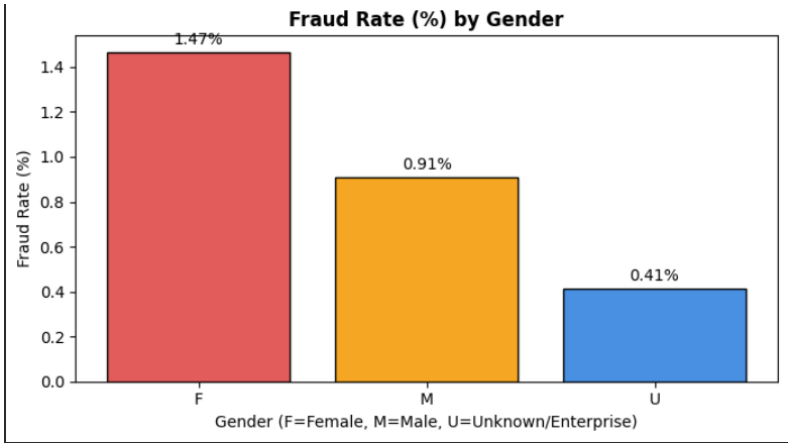

Figure 3: Fraud rate (%) by gender group.

3.4 Fraud Rate by Age Group

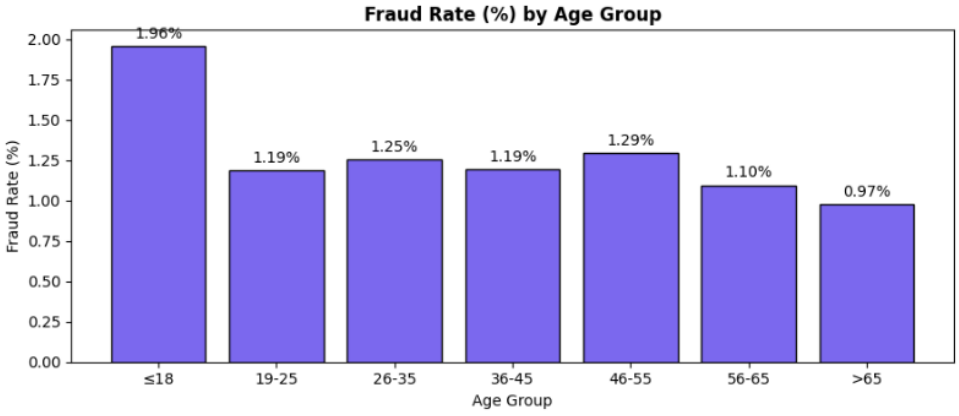

Figure 4: Fraud rate (%) by age group (0 = ≤18, 1 = 19–25, ..., 6 = >65).

Customers aged **18 and under** show the highest fraud rate at **1.96%** — nearly double that of older age groups. This may reflect weaker fraud awareness or security habits among younger users. Older customers (>65) have the lowest fraud rate at 0.97%.

4 Data Preprocessing

The raw dataset contained several issues that were resolved before modeling:

- **Quote-wrapped strings:** All categorical columns stored values with surrounding single quotes were stripped using `str.strip()`.
- **Unknown age values:** The value `'U'` in the age column was replaced with the most frequent age group (mode).
- **Non-standard gender values:** The value `'E'` (Enterprise) was merged into `'U'` (Unknown).
- **Zero-variance columns:** `zipcodeOri` and `zipMerchant` each contained only one unique value across the entire dataset and were dropped.

- **ID columns:** `customer` and `merchant` are unique identifiers with no generalizable signal and were dropped.
- **Encoding:** One-Hot Encoding was applied to `age`, `gender`, and `category`.
- **Feature engineering:** A `time_of_day` feature was extracted from the `step` column (`step % 24`), as fraud patterns vary by hour.
- **Scaling:** The `amount` column was standardized using `StandardScaler` (mean = 0, std = 1).

5 Pipeline

Pipeline

Step	What was done
Preprocessing	Stripped quotes, handled unknowns, dropped zero-variance & ID columns
Feature Engineering	Extracted `time_of_day` from step, OHE for age/gender/category, scaled amount
EDA	Fraud by category, amount, age group, gender, hour of day
Model Training	Logistic Regression, Random Forest, XGBoost — compared on AUC-ROC & F1
Model Interpretation	Feature importance (gain), ROC curves
SMOTE	Synthetic oversampling on training data only (`sampling_strategy=0.1`)
Hyperparameter Tuning	RandomizedSearchCV — 20 iterations, 3-fold CV, scored by AUC-ROC

Figure 5: Pipeline

6 Model Training and Results

Three models were trained on the imbalanced data and evaluated using AUC-ROC and F1-score. Figure 6 shows the comparison.

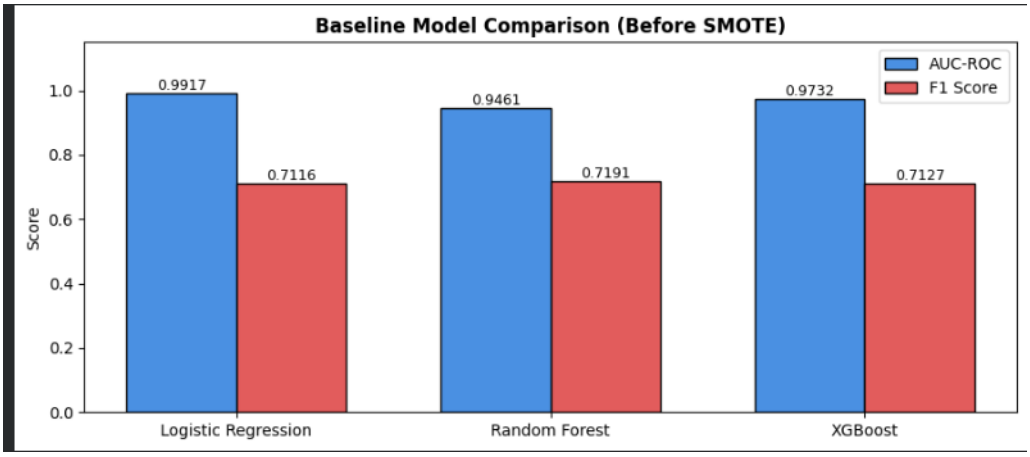

Figure 6: Baseline model comparison before applying SMOTE.

All three models achieve similar scores on imbalanced data. However, the confusion matrix in Figure 7 reveals the real problem — XGBoost misses **553 fraud cases** (recall = 62%).

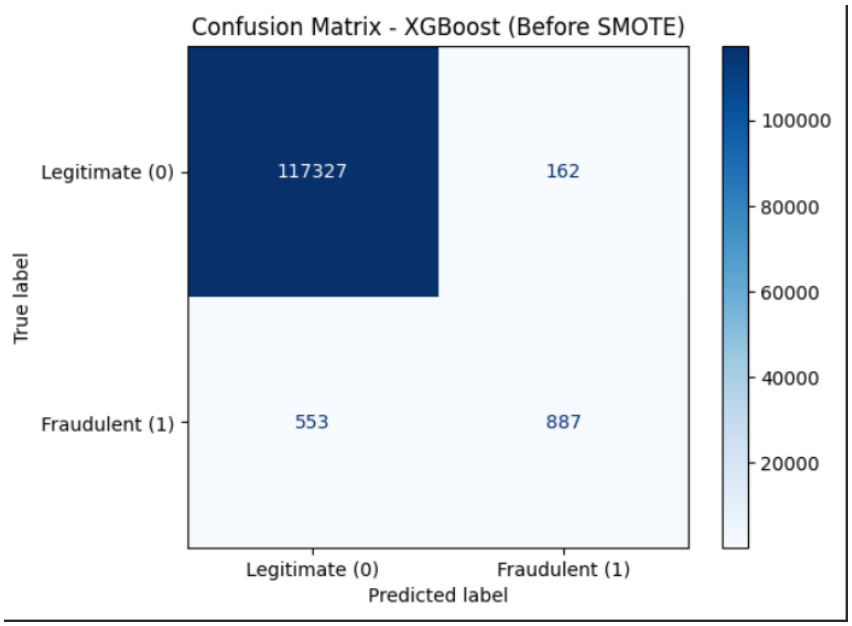

Figure 7: XGBoost confusion matrix before SMOTE. 553 fraud cases missed.

6.1 Handling Class Imbalance with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) was applied exclusively to the training data to generate synthetic fraud samples through interpolation. The test set was never modified, ensuring evaluation reflects real-world distribution.
After applying SMOTE (`sampling_strategy=0.1`), the number of fraud training samples grew from **5,760 to 46,995**. The impact is clear in Figure 8 — missed fraud cases dropped from 553 to **350**, and fraud recall improved from 62% to **76%**.

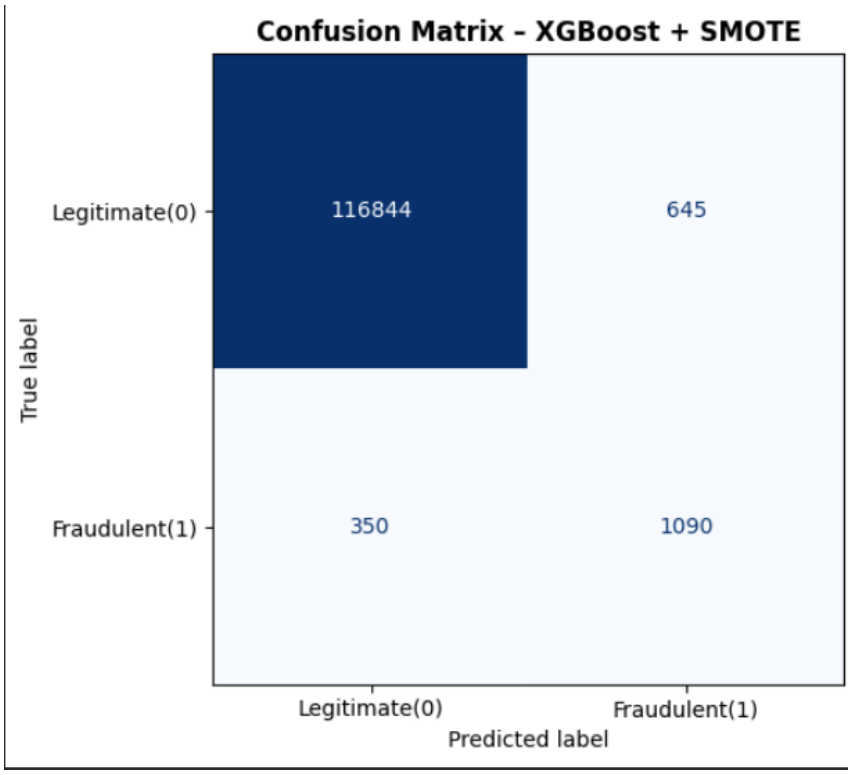

Figure 8: XGBoost confusion matrix after SMOTE. Missed fraud reduced to 350.

6.2 Final Model

Table 2: Full model comparison across all stages.

Model	AUC-ROC	F1	Fraud Recall
Logistic Regression	0.9917	0.71	59%
Random Forest	0.9461	0.72	65%
XGBoost	0.9732	0.71	62%
XGBoost + SMOTE	0.9921	0.69	76%
XGBoost + SMOTE + Tuning	**0.9922**	**0.69**	**76%**

XGBoost + SMOTE + Tuning was selected as the final model. Despite a marginally lower F1 than Logistic Regression, it catches **17% more fraud** (recall 76% vs 59%). For a bank, every missed fraud is a direct financial loss — making recall the priority metric.

7 Model Interpretation

The figure shows the feature importance scores from the final XGBoost model, measured by gain — how much each feature reduces prediction error across all trees.

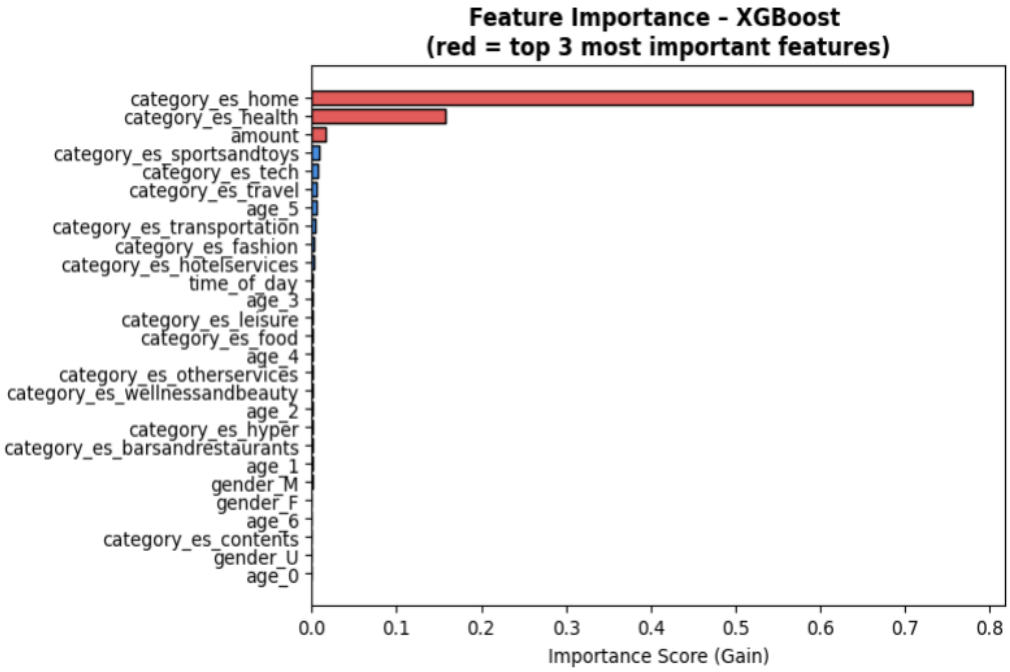

Figure 9: Feature importance scores from the tuned XGBoost model.

`category_es_home` and `category_es_health` are by far the strongest predictors, followed by `amount`. This is notable — while EDA showed `es_leisure` and `es_travel` have the highest raw fraud rates, the model relies more heavily on home and health categories as discriminative signals. Gender and most age features contribute very little to predictions.

8 Key Decisions

- **SMOTE over undersampling:** Undersampling would discard approximately 580,000 legitimate transactions, whereas SMOTE generates synthetic fraud samples through interpolation, preserving the available data.
- **sampling_strategy = 0.1:** Multiple ratios were evaluated. Higher values introduced excessive synthetic samples, which reduced precision and negatively affected overall model performance.
- **No scale_pos_weight with SMOTE:** Using both techniques simultaneously overemphasized the minority class, causing a significant drop in precision and reducing the F1-score from 0.69 to 0.38.
- **RandomizedSearchCV over GridSearchCV:** With seven hyperparameters and multiple candidate values, exhaustive grid search was computationally expensive. RandomizedSearchCV identified near-optimal parameters using only 20 iterations, greatly reducing training time.

9 Business Recommendation

Based on the model's findings, the following actions are recommended:

- **Two-threshold deployment:** Transactions with fraud probability above 0.8 should be auto-blocked. Scores between 0.4 and 0.8 should be flagged for manual review. Below 0.4 should be approved automatically.
- **Category monitoring:** Leisure and travel transactions carry fraud rates above 79% and should receive enhanced real-time scrutiny.
- **Amount-based alerts:** Transaction amount is among the top predictors of fraud. Large or unusual amounts should trigger secondary verification.
- **Regular retraining:** Fraud patterns evolve as fraudsters adapt. The model should be retrained every 3–6 months on fresh labeled data.
- **Feedback loop:** All model predictions and their real outcomes should be logged to continuously improve future model versions.

10 Conclusion

This report presented a complete machine learning pipeline for credit card fraud detection on a severely imbalanced dataset of 594,643 transactions. By cleaning the raw data, analyzing fraud patterns, and testing multiple models, XGBoost combined with SMOTE and hyperparameter tuning achieved an AUC-ROC of 0.9922 and a fraud recall of 76% — catching significantly more fraud than any baseline model. Feature importance revealed which signals the model actually relies on — helping the bank understand and trust its decisions, supporting both model decisions and business monitoring rules.